

Job Scheduling

04 December 2024 14:27

Job scheduling is a feature that allows a user to submit a command or program for execution at a specified time in the future. On a Linux server, it is important that certain tasks run at certain times the execution of the command or program could be one time or periodically based on a predetermined time schedule. For example, scheduling system maintenance commands to run during nonworking hours is a good practice, as it does not disrupt normal business activities

- at – single time execution (non - periodic)
- crontab – periodic execution (google pe crontab garu to check the section)
- anacron – periodic execution (using the desktop)

Field

1. Minutes (0-59)
2. Hours (0-23)
3. Date (1-31)
4. Months (1-12)
5. Day of week (Monday to Sunday) (0-6) or (0-7)

Command

1. date
 - Show the current date and time
2. at
 - at <"time and date"> (current time se aage ka time scheduling karna at the time of compiled the file and dir was created)
 - at> touch /home/file.txt
 - at> mkdir /mnt /demo
 - Save ctrl + d
3. cat /etc/crontab

```
root@ip-172-31-9-221:~# cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
#
```

4. crontab -e

```

root@ip-172-31-9-221:~# crontab -e
no crontab for root - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 2
crontab: installing new crontab
"/tmp/crontab.ja4tCh/crontab":24: bad minute
errors in crontab file, can't install.
Do you want to retry the same edit? (y/n) y
crontab: installing new crontab
"/tmp/crontab.ja4tCh/crontab":24: bad minute
errors in crontab file, can't install.
Do you want to retry the same edit? (y/n) n
crontab: edits left in /tmp/crontab.ja4tCh/crontab
root@ip-172-31-9-221:~# crontab -e
no crontab for root - using an empty one
No modification made

```

5. `!which touch`
6. `apt install at`
-
7. `export EDITOR=vim`
`crontab -e`
- (Change the editor to perform the task)
8. `gpasswd -a vaishali sudo`
- Add the user in sudo group
9. `at crontab defences`

1. Using cron for Recurring Tasks

The cron daemon is used to schedule tasks that repeat periodically.

Edit the Cron Table

- Command:
`crontab -e`
- **Syntax of a Cron Job:**
`* * * * <command_to_run>`
 - The five fields represent:
 - Minute (0–59)
 - Hour (0–23)
 - Day of the month (1–31)
 - Month (1–12)
 - Day of the week (0–6, Sunday = 0 or 7)

Examples:

- Run a script every day at 3:00 AM:
`0 3 * * * /path/to/script.sh`
- Run a command every Monday at 9:15 PM:
`15 21 * * 1 /path/to/command`
- Run a job every 15 minutes:
`* /15 * * * * /path/to/task`

List Current Cron Jobs

- Command:
`crontab -l`

Delete All Cron Jobs

- Command:
`crontab -r`

2. Using at for One-Time Tasks

The at command schedules a job to run once in the future.

Check if at is Installed

- Install if necessary:
sudo apt install at # Debian/Ubuntu
sudo yum install at # RHEL/CentOS

Schedule a Job

- Command:
at <time>

Example:
at 3:00 PM

After entering this command, type the task to be executed, followed by Ctrl+D.

List Scheduled at Jobs

- Command:
atq

Remove a Scheduled Job

- Command:
atrm <job_id>

3. Using systemd timers

systemd timers offer more robust and modern job scheduling options.

Create a Timer Unit

- Example: Create mytask.timer in /etc/systemd/system/
[Unit]
Description=Run My Task Periodically

[Timer]
OnCalendar=daily

[Install]
WantedBy=timers.target

Create a Service File

- Example: Create mytask.service in /etc/systemd/system/
[Unit]
Description=Run My Task

[Service]
ExecStart=/path/to/script.sh

Enable and Start the Timer

- Command:
sudo systemctl enable mytask.timer
sudo systemctl start mytask.timer

Check Timer Status

- Command:
systemctl list-timers

4. Additional Tools

- **anacron:**
Used for systems that are not always running. Ensures tasks are executed even if the system was off at the scheduled time.
- **Manual Scheduling:**
Use sleep and & for ad-hoc jobs:
sleep 3600 && /path/to/command &

Comparison of Tools

Feature	cron	at	systemd timers
Recurring	Yes	No	Yes
One-Time	No	Yes	Yes
Dependency Management	Limited	No	Excellent
User-Friendly	Moderate	Easy	Moderate